Retail E-Commerce Sales Analytics Pipeline

CEI Intern - Final Project Report

Prepared by: Saksham Sharma
ID - CT_CSI_DE_519
Submitted to: Celebal Technologies

Table of Contents

Section	Description
1. Executive Summary	Overview of the retail e-commerce data pipeline built on Databricks using Medallion Architecture.
2. Problem Statement	Ingestion and processing of dirty retail data, including schema changes and historical tracking requirements.
3. Tech Stack	Tools used: Databricks, PySpark, Delta Lake, Unity Catalog, Auto Loader, and Spark SQL.
4. Architecture Overview	Explanation of the five-stage Medallion Architecture flow from raw data to gold analytics.
5. Notebook-by-Notebook Implementation	Detailed breakdown of the five project notebooks, cell by cell.
5.1 00_setup.ipynb - Environment Setup	Configuration of catalogs, schemas, volumes, and file paths.
5.2 01_bronze.ipynb - Raw Ingestion	Raw ingestion into Delta tables with audit columns and incremental processing.
5.3 02_silver_stage1.ipynb - Clean, Cast, Deduplicate, Quarantine	Data cleaning, type casting, deduplication, and handling of invalid records.
5.4 03_silver_stage2_scd2.ipynb - SCD Type 2 Dimensions	Building customer and product dimensions with historical tracking.
5.5 04_gold.ipynb - Conformed Fact Table & Analytics Tables	Creation of the conformed fact table and summary reporting tables.
6. Data Quality Results	Summary of data cleaning results and analysis of unmatched joins.
7. Delta Lake Features Demonstrated	Demonstration of ACID transactions, table history, time travel, and schema evolution.
8. Final Deliverables	Overview of produced Delta tables across raw, silver, and gold schemas.
9. How to Reproduce	Steps to reproduce the project using the provided dataset and notebook execution order.
10. Conclusion	Summary of project achievements and data engineering diligence.

1. Executive Summary

This report documents the design, implementation, and results of an end-to-end retail e-commerce data pipeline built on Databricks using PySpark and Delta Lake, developed as part of the CEI Internship program at Celebal Technologies. The pipeline follows the **Medallion Architecture** pattern **(Bronze → Silver → Gold)** with an additional Slowly Changing Dimension Type 2 (SCD2) stage for historical tracking of customer and product attributes.

The objective was to take realistic, deliberately dirty retail data - historical batch files plus daily incremental/CDC feeds - and turn it into clean, trustworthy, analytics-ready tables, while correctly preserving the history of changing customer and product attributes and gracefully handling a mid-stream schema change.

The pipeline is implemented as five Databricks notebooks, executed end-to-end on a fresh cluster and validated with row-count checks, sample queries, and Delta Lake's history/time-travel features. Section 5 walks through every notebook cell by cell, in plain language, with the exact verification screenshot expected at each step.

2. Problem Statement

An e-commerce company maintains historical order, customer, product, and store data, and receives new orders and CDC (Change Data Capture) files every day. The raw data contains null customer/product IDs, duplicate order records, invalid or unparseable dates, numeric fields stored as text (e.g. "unknown" prices), updated customer/product attributes that need historical tracking, late-arriving orders, and a schema change — a new coupon_code column appearing partway through the incremental feed (2026-04-26).

The task was to ingest this data reliably, clean and validate it without silently discarding anything, track the full history of customer and product attribute changes using SCD Type 2, and produce a conformed, analytics-ready fact table plus summary reporting tables - while demonstrating Delta Lake's ACID transactions, schema evolution, table history, and time travel.

3. Tech Stack

Tool / Technology	Role in this project
Databricks (Free Edition)	Workspace, notebooks, and compute used to build and run the entire pipeline
PySpark	Primary language for all data transformation, cleaning, and aggregation logic
Delta Lake	Storage format for every table — ACID transactions, schema evolution, table history, time travel
Unity Catalog	Organizes all tables under retail_demo.raw / .silver / .gold and hosts the raw file volume
Auto Loader (cloudFiles)	Incrementally ingests new daily CSV files with automatic schema evolution
Spark SQL	Verification queries, DESCRIBE HISTORY, and time-travel queries

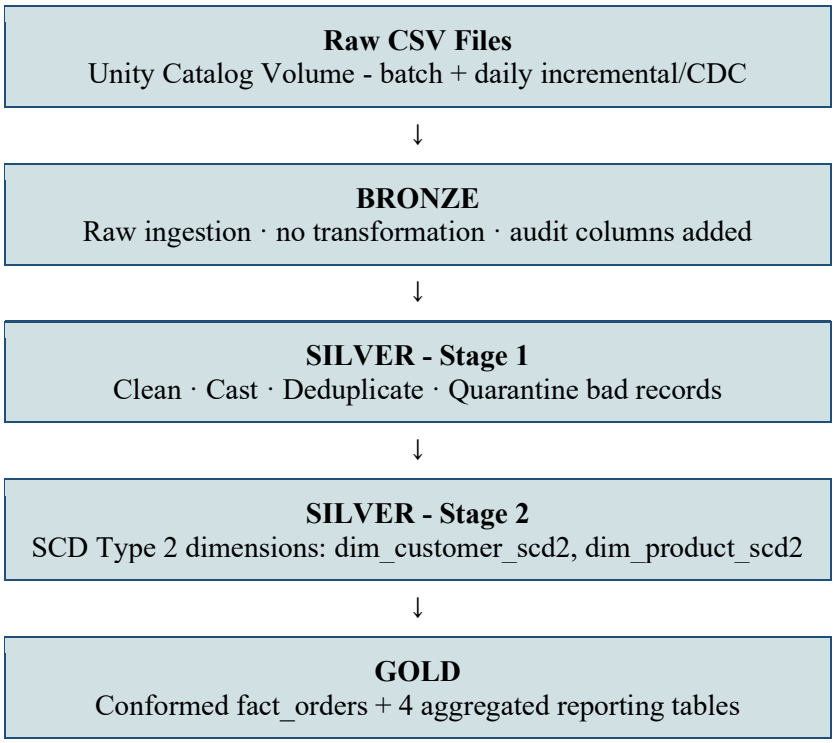
4. Architecture Overview

The pipeline follows a five-stage Medallion Architecture:

- Bronze - raw ingestion of batch and incremental files, no transformation, all columns kept as strings, with audit metadata added.

- Silver Stage 1 - type casting, data cleaning, deduplication, and quarantine of invalid records.
- Silver Stage 2 - SCD Type 2 dimension tables for customers and products, tracking every historical attribute change.
- Gold - a conformed fact table joined to the SCD2 dimensions via point-in-time logic, plus four aggregated reporting tables.

4.1 Data Flow Diagram



5. Notebook-by-Notebook Implementation

Every notebook is broken down below exactly as it is structured in the workspace - cell by cell - explaining in plain points what each cell does and why, with the exact verification screenshot expected at that point.

5.1 00_setup.ipynb - Environment Setup

Foundation notebook that every other notebook imports via %run "./00_setup". Has 2 cells.

Cell 1 - Create Catalog, Schemas, and Volume

- Defines the 4 name constants used everywhere in the project: CATALOG, RAW_SCHEMA, SILVER_SCHEMA, GOLD_SCHEMA.
- Runs CREATE CATALOG / CREATE SCHEMA / CREATE VOLUME IF NOT EXISTS for retail_demo and its raw, silver, gold schemas.
- IF NOT EXISTS makes the cell idempotent - safe to re-run every time this notebook is imported by another notebook.

```
CATALOG = "retail_demo"
RAW_SCHEMA = "raw"
SILVER_SCHEMA = "silver"
GOLD_SCHEMA = "gold"

spark.sql(f"CREATE CATALOG IF NOT EXISTS {CATALOG}")
spark.sql(f"CREATE SCHEMA IF NOT EXISTS {CATALOG}.{RAW_SCHEMA}")
spark.sql(f"CREATE SCHEMA IF NOT EXISTS {CATALOG}.{SILVER_SCHEMA}")
spark.sql(f"CREATE SCHEMA IF NOT EXISTS {CATALOG}.{GOLD_SCHEMA}")
spark.sql(f"CREATE VOLUME IF NOT EXISTS {CATALOG}.{RAW_SCHEMA}.retail_files")

print("Catalog and schemas created successfully")
```

> See performance (5) ✓ Checked for improvements

```
Catalog and schemas created successfully
```

Cell 2 - Path Configuration

- Defines BATCH_PATH and INCR_PATH - where the batch and incremental CSVs live inside the Unity Catalog volume.
- Defines CHECKPOINT_PATH and SCHEMA_PATH - used later by Auto Loader (01_bronze) to track streaming progress and schema evolution history.
- Centralizing paths here means no other notebook hardcodes a path independently.

```
BASE_PATH = "/Volumes/retail_demo/raw/retail_files/datasets"

BATCH_PATH = f"{BASE_PATH}/batch"
INCR_PATH = f"{BASE_PATH}/incremental"
CHECKPOINT_PATH = "/Volumes/retail_demo/raw/retail_files/_checkpoints"
SCHEMA_PATH = "/Volumes/retail_demo/raw/retail_files/_schemas"

print("BATCH_PATH:", BATCH_PATH)
print("INCR_PATH:", INCR_PATH)
```

```
BATCH_PATH: /Volumes/retail_demo/raw/retail_files/datasets/batch
INCR_PATH: /Volumes/retail_demo/raw/retail_files/datasets/incremental
```

5.2 01_bronze.ipynb - Raw Ingestion

Lands every source file into Delta tables with zero transformation, so a malformed row can never fail the ingestion job. Bronze is a safe, replayable archive of the raw source.

Cell 1 - Import Setup

- Runs **%run "./00_setup"** to bring CATALOG, BATCH_PATH, INCR_PATH etc. into this notebook's session.

Cell 2 - Audit Column Helper

- Defines add_audit_cols(), a reusable function that stamps 3 metadata columns onto any DataFrame before it's written.
- source_file - the exact input file a row came from (pulled from Spark's hidden _metadata.file_path).
- ingestion_ts - the timestamp the row was processed.
- load_type - "batch" or "incremental", so every table can be filtered by how the row arrived.

Cell 3 - Batch Ingestion (4 tables)

- Loops over the 4 historical batch files: orders, customers, products, stores.
- Reads each with inferSchema = False - every column forced to string, so a dirty value like "unknown" in a price column can never crash the read.

- Adds audit columns, then writes as a Delta table in the raw schema with mode("overwrite") - safe to re-run.

```
> df: pyspark.sql.connect.dataframe.DataFrame = [store_id: string, store_name: string ... 6 more fields]
retail_demo.raw.bronze_orders: 12180 rows loaded
retail_demo.raw.bronze_customers: 2560 rows loaded
retail_demo.raw.bronze_products: 830 rows loaded
retail_demo.raw.bronze_stores: 80 rows loaded
```

Cell 4 - Incremental Ingestion via Auto Loader (3 tables)

- Daily orders/customers/products CDC files are read using spark.readStream.format("cloudFiles") instead of a plain batch read.
- Auto Loader automatically discovers new files under INCR_PATH - no manual tracking of what's already been processed.
- cloudFiles.schemaEvolutionMode = "addNewColumns" + mergeSchema = "true" on write together let the coupon_code column (only in the 2026-04-26 file) get absorbed automatically instead of throwing a schema-mismatch error.
- trigger(availableNow=True) runs this as a single micro-batch - process everything currently available, then stop - suited to an on-demand pipeline rather than a 24/7 stream.

```
> stream_df: pyspark.sql.connect.dataframe.DataFrame = [product_id: string, product_name: string ... 11 more fields]
retail_demo.raw.bronze_orders_incremental done -> 6135 rows
retail_demo.raw.bronze_customers_cdc done -> 630 rows
retail_demo.raw.bronze_products_cdc done -> 180 rows
```

Cell 5 - Sanity Check: Preview All 7 Bronze Tables

- Loops over all 7 bronze tables and displays 3 sample rows from each, to visually confirm data landed correctly and audit columns are populated.

```
--- bronze_orders ---
```

Table

	order_id	order_ts	customer_id	product_id	store_id	quantity	unit_price	discount_pct	
1	O0000001	2025-12-15 04:49:...	C01671	P00638	S003	4	30260.29	0.15	12
2	O0000002	2026-03-13 03:29:...	C00713	P00768	S028	4	30049.26	0.1	10
3	O0000003	2026-03-05 07:20:...	C02131	P00142	S072	5	59685.88	0.0	28

```
--- bronze_customers ---
```

Table

	customer_id	customer_name	city	segment	gender	signup_date	status	source_file
1	C00001	Customer_1	Delhi	Regular	Other	2024-10-08	active	dbfs:/Volumes/retail_demo/raw
2	C00002	Customer_2	Bengaluru	Silver	Other	2024-04-14	active	dbfs:/Volumes/retail_demo/raw
3	C00003	Customer_3	Gurugram	Platinum	M	2024-01-31	active	dbfs:/Volumes/retail_demo/raw

3 rows | 10.572s runtime

Refreshed yesterday

--- bronze_products ---

Table

	product_id	product_name	category	brand	unit_price	status	created_date	source_file
1	P00001	T-Shirt 1	Fashion	BrandC	unknown	discontinued	2025-02-15	dbfs:/Volumes/retail_demo/
2	P00002	Bedsheet 2	Home	BrandC	unknown	active	2024-04-20	dbfs:/Volumes/retail_demo/
3	P00003	Bedsheet 3	Home	BrandD	30753.26	active	2024-10-23	dbfs:/Volumes/retail_demo/

3 rows | 10.572s runtime

Refreshed yesterday

--- bronze_stores ---

Table

	sto...	store_name	city	region	status	source_file	ingesti
1	S001	Store_1	Jaipur	Online	active	dbfs:/Volumes/retail_demo/raw/retail_files/datasets/batch/stores_batch.csv	2026-07-1
2	S002	Store_2	Pune	West	active	dbfs:/Volumes/retail_demo/raw/retail_files/datasets/batch/stores_batch.csv	2026-07-1
3	S003	Store_3	Ahmedabad	South	closed	dbfs:/Volumes/retail_demo/raw/retail_files/datasets/batch/stores_batch.csv	2026-07-1

3 rows | 10.572s runtime

Refreshed yesterday

--- bronze_orders_incremental ---

Table

	order_id	order_ts	customer_id	product_id	store_id	quantity	unit_price	discount_pct	
1	OI3000001	2026-04-26 22:42:...	C01830	P00296	S056	5	64201.56	0.1	25
2	OI3000002	2026-04-26 04:25:...	C01705	P00334	S030	2	16032.79	0.0	25
3	OI3000003	2026-04-23 00:00:...	C01378	P00014	S045	6	24907.63	0.1	14

3 rows | 10.572s runtime

Refreshed yesterday

--- bronze_customers_cdc ---

Table

	customer_id	customer_name	city	segment	gender	signup_date	status	effective_date	
1	C00606	Customer_606	Kolkata	Silver	Other	2025-10-09	active	not_a_date	UP
2	C00527	Customer_527	Bengaluru	Regular	M	2024-04-10	inactive	2026-04-25	UP
3	C00660	Customer_660	Jaipur	Regular	F	2024-10-16	active	2026-04-25	UP

3 rows | 10.572s runtime

Refreshed yesterday

--- bronze_products_cdc ---

Table

	product_id	product_name	category	brand	unit_price	status	created_date	effective_date
1	P00173	Speaker 173	Electronics	BrandA	50560.38	discontinued	2025-09-11	2026-04-25
2	P00680	Perfume 680	Beauty	BrandC	40368.75	active	2024-12-24	2026-04-25
3	P00095	Smartphone 95	Electronics	BrandB	685.97	active	2023-10-15	2026-04-25

3 rows | 10.572s runtime

Refreshed yesterday

5.3 02_silver_stage1.ipynb - Clean, Cast, Deduplicate, Quarantine

This is where data quality is actually enforced. Every entity is cast to proper types, cleaned, deduplicated, and split into a clean table plus a quarantine table - bad rows are never simply deleted.

Cell 1 - Import Setup

- Run **%run "./00_setup"**.

Cell 2 - Helper Functions

- dedup_latest() - window function partitioned by the business key, ordered by ingestion_ts descending, keeps only row_number = 1 (the most recently ingested version).
- clean_numeric() - regex strips every character that isn't a digit, decimal point, or minus sign (turns "unknown" or "$45.00" into a clean number or null), then casts to double.

Cell 3 - Orders: Union, Clean, Cast, Quarantine

- Batch + incremental orders combined with unionByName(allowMissingColumns=True), after aligning columns that don't exist in both (coupon_code only in incremental; ingest_date only in batch).
- order_ts parsed with try_to_timestamp — malformed values become null instead of failing the job.
- unit_price and gross_amount cleaned via clean_numeric().
- Rows missing order_id, customer_id, product_id, order_ts, unit_price, or gross_amount → routed to orders_quarantine.
- Remaining rows become orders_clean, deduplicated on order_id via dedup_latest().

```
> orders_batch:  pyspark.sql.connect.dataframe.DataFrame = [order_id: string, order_ts: string ... 14 more fields]
> orders_clean:  pyspark.sql.connect.dataframe.DataFrame = [order_id: string, order_ts: timestamp ... 15 more fields]
> orders_incr:  pyspark.sql.connect.dataframe.DataFrame = [order_id: string, order_ts: string ... 15 more fields]
> orders_quarantine:  pyspark.sql.connect.dataframe.DataFrame = [order_id: string, order_ts: timestamp ... 15 more fields]
> orders_raw:  pyspark.sql.connect.dataframe.DataFrame = [order_id: string, order_ts: string ... 15 more fields]
> orders_typed:  pyspark.sql.connect.dataframe.DataFrame = [order_id: string, order_ts: timestamp ... 15 more fields]
orders clean: 17273  | quarantined: 734
```

Cell 4 - Customers: Clean, Cast, Quarantine

- signup_date parsed with try_to_date.
- Missing categorical fields (city, segment, gender) filled with "Unknown" rather than dropped - not critical fields, so the row is kept and flagged instead of lost.
- Only a missing/blank customer_id causes quarantine, since it's the field every downstream join and SCD2 key depends on.

```
> customers_batch:  pyspark.sql.connect.dataframe.DataFrame = [customer_id: string, customer_name: string ... 8 more fields]
> customers_clean:  pyspark.sql.connect.dataframe.DataFrame = [customer_id: string, customer_name: string ... 8 more fields]
> customers_quarantine:  pyspark.sql.connect.dataframe.DataFrame = [customer_id: string, customer_name: string ... 8 more fields]
> customers_typed:  pyspark.sql.connect.dataframe.DataFrame = [customer_id: string, customer_name: string ... 8 more fields]
customers clean: 2500  | quarantined: 0
```

Cell 5 - Products: Clean, Cast, Quarantine

- Same clean_numeric() treatment on unit_price (handles "unknown" values).
- created_date parsed; category/brand null-filled with "Unknown".

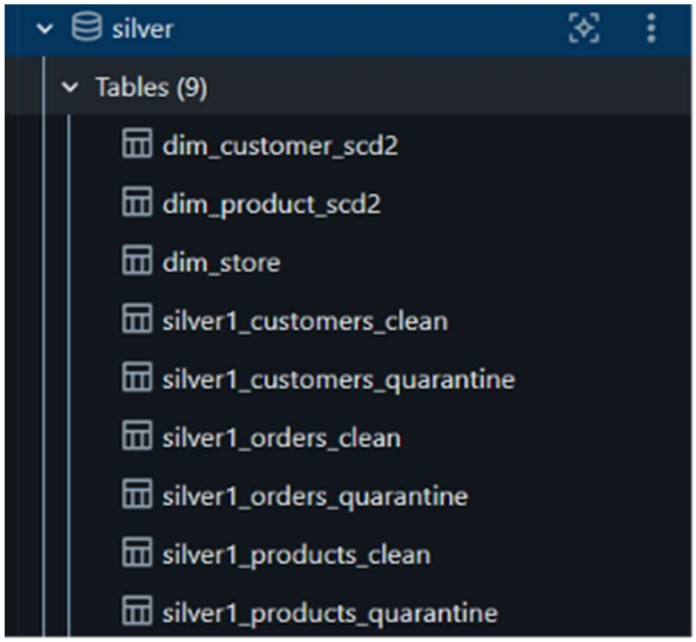
- Quarantined if product_id is missing or unit_price couldn't be parsed at all.

```
products_batch: pyspark.sql.connect.dataframe.DataFrame = [product_id: string, product_name: string ... 8 more fields]
products_clean: pyspark.sql.connect.dataframe.DataFrame = [product_id: string, product_name: string ... 8 more fields]
products_quarantine: pyspark.sql.connect.dataframe.DataFrame = [product_id: string, product_name: string ... 8 more fields]
products_typed: pyspark.sql.connect.dataframe.DataFrame = [product_id: string, product_name: string ... 8 more fields]
products clean: 780  | quarantined: 20
```

Cell 6 - Stores: Deduplicate

- Small, mostly-clean reference table - just deduplicated on store_id via dedup_latest() and written directly as dim_store. No separate quarantine table needed.

```
stores_batch: pyspark.sql.connect.dataframe.DataFrame = [store_id: string, store_name: string ... 6 more fields]
stores_clean: pyspark.sql.connect.dataframe.DataFrame = [store_id: string, store_name: string ... 6 more fields]
stores clean: 75
```



5.4 03_silver_stage2_scd2.ipynb - SCD Type 2 Dimensions

Builds dim_customer_scd2 and dim_product_scd2 - the two dimensions the business needs full history for. Every cell exists to answer one question: "what did this customer/product look like on the exact date an order was placed?"

Cell 1 - Import Setup

- Runs **%run "./00_setup"**.

Cell 2 - Row Hash Helper

- row_hash() concatenates all tracked attribute columns into one pipe-delimited string (nulls coalesced to "NA") and runs it through SHA-256.
- Turns "did any of these columns change?" into one string comparison instead of comparing every column individually.

Cell 3 - Initial Load: Customer Dimension

- Takes silver1_customers_clean and gives every customer a starting SCD2 row.
- hash_value computed from customer_name, city, segment, gender, status.

- effective_start_date = signup_date; effective_end_date = far-future (9999-12-31); is_current = true.
- customer_sk generated as SHA-256 of customer_id + effective_start_date - a unique, deterministic key for every historical version.

```
customers_clean: pyspark.sql.connect.dataframe.DataFrame = [customer_id: string, customer_name: string ... 8 more fields]
dim_customer_init: pyspark.sql.connect.dataframe.DataFrame = [customer_sk: string, customer_id: string ... 10 more fields]
initial customer dim rows: 2500
```

Cell 4 - Initial Load: Product Dimension

- Identical pattern applied to silver1_products_clean - tracks product_name, category, brand, unit_price, status; keyed on created_date instead of signup_date.

```
dim_product_init: pyspark.sql.connect.dataframe.DataFrame = [product_sk: string, product_id: string ... 10 more fields]
products_clean: pyspark.sql.connect.dataframe.DataFrame = [product_id: string, product_name: string ... 8 more fields]
initial product dim rows: 780
```

Cell 5 - apply_scd2() - The Two-Step MERGE Function

Core SCD2 engine, written once and reused for both customers and products:

- Processes CDC data one effective_date at a time, oldest-first, so history reconstructs correctly even with late-arriving records.
- For each date, joins the day's incoming rows against the currently active (is_current = true) dimension rows on the business key.
- Keeps only rows that are brand new, or whose hash differs from the active row's hash - i.e. genuinely changed records.
- If nothing changed that day, skips straight to the next date.
- Step 1 (expire): MERGE that sets is_current = false and effective_end_date = day before the change, on the currently active row.
- Step 2 (insert): a new row is built with a fresh surrogate key, the new attribute values, and is_current = true, then inserted via a second MERGE.

Cell 6 - Apply Customer CDC

- Reads bronze_customers_cdc, parses effective_date and signup_date.
- dropDuplicates(["customer_id", "effective_date"]) - if the same change appears twice in a day's file, it's only applied once.
- Rows with an unparseable effective_date are dropped (can't be placed in the timeline).
- Passes the cleaned CDC stream into apply_scd2() against dim_customer_scd2.

```
customers_cdc: pyspark.sql.connect.dataframe.DataFrame = [customer_id: string, customer_name: string ... 11 more fields]
retail_demo.silver.dim_customer_scd2: applied 0 changes for effective_date=2026-04-24
retail_demo.silver.dim_customer_scd2: applied 0 changes for effective_date=2026-04-25
retail_demo.silver.dim_customer_scd2: applied 0 changes for effective_date=2026-04-26
```

Cell 7 - Apply Product CDC

- Same pattern as customers, with an added try_cast(unit_price AS DOUBLE) since bronze data always arrives as string.

```
products_cdc: pyspark.sql.connect.dataframe.DataFrame = [product_id: string, product_name: string ... 11 more fields]
retail_demo.silver.dim_product_scd2: applied 0 changes for effective_date=2026-04-24
retail_demo.silver.dim_product_scd2: applied 0 changes for effective_date=2026-04-25
retail_demo.silver.dim_product_scd2: applied 0 changes for effective_date=2026-04-26
```

Cell 8 - Sanity Check: Confirm History Is Actually Being Tracked

- Prints total row count in both dimension tables - should now exceed the initial load counts, since every changed entity has 2+ rows.
- Runs groupBy("customer_id").count().filter("count > 1") to list every customer with more than one historical version - direct proof the two-step MERGE created new rows instead of overwriting in place.

```
Total customer dim rows: 3093
Total product dim rows: 960
```

	customer_id	count
1	C00175	2
2	C00552	3
3	C00929	3
4	C01257	2
5	C01414	2
6	C01426	2
7	C01684	2
8	C01937	2
9	C02069	2
10	C02309	3
11	C00186	2
12	C00423	2
13	C01042	2
14	C01177	2
15	C01252	2

336 rows | 2.228s runtime — Refreshed yesterday

Cell 9 - Customer SCD2 History Example

_sqldf: pyspark.sql.connect.dataframe.DataFrame = [customer_sk: string, customer_id: string ... 10 more fields]

	customer_sk	customer_id	customer_name	city	segment	gender
1	9e5caf29ba7ca09cfe86d9cde21c98a9b759e4a1cda78cd564cb530be59da4d8	C00175	Customer_175	Gurugram	Regular	Other
2	34b18aa3bc8fc20d4456654397be7340564efb32b030b797936fefa9e993b330	C00175	Customer_175	Chennai	Platinum	Other

2 rows | 4.075s runtime — Refreshed 1 minute ago

Cell 10 - Product SCD2 History Example

_sqldf: pyspark.sql.connect.dataframe.DataFrame = [product_sk: string, product_id: string ... 10 more fields]

	product_sk	product_id	product_name	category	brand	unit_price
1	0654595a11776980957cb5342ba2f4672d0ab072f5064f6758933636b535eb...	P00671	Perfume 671	Beauty	BrandC	154
2	93d263a816a8ce1c5678ace3ad3d8e35f4dd7e5a279ef1509cf9ac38f64f9282	P00671	Perfume 671	Beauty	BrandC	78823

2 rows | 4.235s runtime — Refreshed 1 minute ago

5.5 04_gold.ipynb - Conformed Fact Table & Analytics Tables

Brings orders and all three dimensions together into a single, analysis-ready fact table, then builds the four reporting tables on top of it.

Cell 1 - Import Setup

- Runs **%run "./00_setup"**.

Cell 2 - Load Silver Tables

- Pulls in silver1_orders_clean and all three dimensions (dim_customer_scd2, dim_product_scd2, dim_store), printing each one's row count as a pre-join baseline.

```
> dim_customer:  pyspark.sql.connect.dataframe.DataFrame = [customer_sk: string, customer_id: string ... 10 more fields]
> dim_product:  pyspark.sql.connect.dataframe.DataFrame = [product_sk: string, product_id: string ... 10 more fields]
> dim_store:  pyspark.sql.connect.dataframe.DataFrame = [store_id: string, store_name: string ... 6 more fields]
> orders:  pyspark.sql.connect.dataframe.DataFrame = [order_id: string, order_ts: timestamp ... 15 more fields]

orders: 17273
dim_customer: 3093
dim_product: 960
dim_store: 75
```

Cell 3 - Build fact_orders (Point-in-Time Join)

- order_ts truncated to order_date.
- Left-joined to dim_customer_scd2 on customer_id AND order_date between effective_start_date and effective_end_date - the point-in-time join that resolves customer_sk to the customer's state at the time of the order, not today's state.
- Same point-in-time join pattern applied for dim_product_scd2 to resolve product_sk.
- Left-joined to dim_store on store_id directly (stores have no SCD2 history in this project).
- Result written to gold.fact_orders.

```
> fact_orders:  pyspark.sql.connect.dataframe.DataFrame = [order_id: string, order_date: date ... 18 more fields]
> orders_d:  pyspark.sql.connect.dataframe.DataFrame = [order_id: string, order_ts: timestamp ... 16 more fields]

fact_orders rows: 17273
```

_sqldf: pyspark.sql.connect.dataframe.DataFrame = [order_id: string, order_date: date ... 18 more fields]

Table +

	order_id	order_date	order_ts	customer_sk	customer_id	product_sk
1	O0000001	2025-12-15	2025-12-15T04:49:00.000+00:...	46695eac27bcfc1cd6ef628c1cd9e906ba842f332bcad364d40b1a2930606679	C01671	87eb8dc90d0c0ff0ed1
2	O0000004	2025-12-28	2025-12-28T13:56:00.000+00:...	cd9084d058117a20ef23850a55eb353614492d94f319a47b886ab56e416e83d0	C01399	b1821d99d372f76873
3	O0000005	2025-12-31	2025-12-31T17:43:00.000+00:...	8b0314b89d7d52d1b91cd7805372bdf7fca4a59f9237609770db67a8c54604d4	C00383	166185d29b78aa61d6
4	O0000007	2025-12-26	2025-12-26T03:16:00.000+00:...	f29cce2a246590ee8f95c0c30a10fc4295d492e82aff010088ab762e12c0acf3	C01633	9c5ac2d00f8f97a87255
5	O0000009	2026-01-20	2026-01-20T19:25:00.000+00:...	1c5cecc5c68c51e72255ca03b6463ab2a81488d988e8ef32380e495d4cefa4cf	C00479	62f086ff842cb3173a4c
6	O0000010	2026-03-21	2026-03-21T06:52:00.000+00:...	34d1d7517c9e09bc074bea836393325aad961cf764cecb37af417ac335495cd7	C00197	c8233df98d552c26f93
7	O0000011	2025-12-15	2025-12-15T20:38:00.000+00:...	086fb5969aafc8d4c1c130b4947aabb5e0c0e581d3247b1e5be062a5dec8dbfd	C00223	559d67241963ee7a2a
8	O0000012	2026-01-22	2026-01-22T20:26:00.000+00:...	8671cdcc736686cf24cbcca6b79dda37a66f89ad123e9b885aaa70a67ded80a8	C00904	72b94345f46df3faa98
9	O0000015	2025-11-11	2025-11-11T18:44:00.000+00:...	14d6c577d33d47cb8563854e87475785d1845a77ecb59e801361fe101b636d...	C00094	085e35b24b2c6c24ab
10						

10 rows | 13.849s runtime Refreshed 2 minutes ago

Cell 4 - Build the 4 Gold Analytics Tables

- gold_daily_sales - orders, revenue, units, and average order value grouped by order_date.
- gold_category_sales - orders, revenue, units grouped by product category, sorted by revenue descending.
- unique customers (via countDistinct), orders, and revenue grouped by customer segment.
- gold_region_sales - orders, revenue, units grouped by store region.

Table +

	order_date	total_orders	total_revenue	total_units	avg_order_value
1	2025-10-01	60	7931198.779999998	193	132186.65
2	2025-10-02	49	5833780.93	135	119056.75
3	2025-10-03	57	6742787.449999999	160	118294.52
4	2025-10-04	64	8789390.049999999	201	137334.22
5	2025-10-05	53	6658672.449999999	159	125635.33
6	2025-10-06	61	8034282.170000002	188	131709.54
7	2025-10-07	63	8086661.39	174	128359.7
8	2025-10-08	63	6900786.519999998	185	109536.29
9	2025-10-09	49	6025594.72	136	122971.32
10	2025-10-10	65	7819608.3199999975	208	120301.67
11	2025-10-11	55	5819227.76	162	105804.14
12	2025-10-12	59	6982595.3	169	118349.07
13	2025-10-13	75	10185263.930000002	222	135803.52
14	2025-10-14	55	7220497.740000001	168	131281.78
15	2025-10-15	67	7994526.57	200	119321.29

208 rows | 21.681s runtime

Table

	category	total_orders	total_revenue	total_units
1	Home	3822	482369020.0500017	12096
2	Fashion	3599	463269670.4999988	11275
3	Grocery	2999	396689875.9199998	9355
4	Electronics	3120	395295560.8099997	9790
5	Beauty	2901	366172440.8300001	9089
6	null	548	67951960.89999993	1633
7	Unknown	284	37163352.42	865

7 rows | 21.681s runtime

Table

	segment	unique_customers	total_orders	total_revenue
1	Platinum	689	4298	547811450.3299999
2	Regular	662	4222	540057267.8199997
3	Silver	648	4110	527014138.31999964
4	Gold	657	3923	505890842.35000074
5	null	283	720	88138182.61000003

5 rows | 21.681s runtime

Table

	region	total_orders	total_revenue	total_units
1	Online	4408	560907100.0899991	13667
2	South	3950	507050415.9600001	12365
3	North	3295	428944826.7199995	10451
4	West	3177	407807843.6800004	10074
5	East	2443	304201694.9800008	7546

5 rows | 21.681s runtime

Cell 5 - Delta Lake Features: Table History

- Runs DESCRIBE HISTORY retail_demo.gold.fact_orders to show the full transaction log.

_sqldf: pyspark.sql.connect.dataframe.DataFrame = [version: long, timestamp: timestamp ... 14 more fields]

Table

	version	timestamp	userId	userName	operation	operationParameters
1	1	2026-07-13T10:48:16.000+00:...	782022865192...	nanus560.ns@gmail.co...	CREATE OR REPLACE TABLE AS SELECT	{"isV1SaveAsTableOverwrite":"true","partitionBy":"[]","clusterBy":"[
2	0	2026-07-12T09:13:57.000+00:...	782022865192...	nanus560.ns@gmail.co...	CREATE OR REPLACE TABLE AS SELECT	{"isV1SaveAsTableOverwrite":"true","partitionBy":"[]","clusterBy":"[

2 rows | 3.740s runtime

Refreshed now

Cell 6 - Delta Lake Features: Time Travel

- Reads fact_orders at versionAsOf 0 (its first write) and compares the row count against the latest version - a runnable proof that Delta Lake retains and can query any historical version of a table.

```
> df_latest: pyspark.sql.connect.dataframe.DataFrame = [order_id: string, order_date: date ... 18 more fields]
> df_v0: pyspark.sql.connect.dataframe.DataFrame = [order_id: string, order_date: date ... 18 more fields]
Version 0 row count: 17273
Latest version row count: 17273
```

Cell 7 - Schema Evolution (Reference Note)

- Markdown note pointing back to 01_bronze: coupon_code was already absorbed automatically at ingestion time via Auto Loader's schema evolution - visible as null on day-1/day-2 rows and populated on day-3 rows.

Cell 8 - Final Sanity Check

- Counts fact_orders rows with a null customer_sk, null product_sk, or null store_id (the unmatched-join diagnostic covered in Section 6).
- Prints total gross_amount across the fact table as a plausibility check.
- Previews all 4 gold tables once more in a single combined cell.

```
orders with missing customer_sk: 720
orders with missing product_sk: 536
orders with missing store: 0

total gross_amount in fact_orders: 2208911881.4300117

--- gold_daily_sales ---
```

Table +

	order_date	total_orders	total_revenue	total_units	avg_order_value
1	2025-10-01	60	7931198.779999998	193	132186.65
2	2025-10-02	49	5833780.93	135	119056.75
3	2025-10-03	57	6742787.449999999	160	118294.52
4	2025-10-04	64	8789390.049999999	201	137334.22
5	2025-10-05	53	6658672.449999999	159	125635.33

5 rows | 7.914s runtime

```
--- gold_category_sales ---
```

Table +

	category	total_o...	total_revenue	total_units
1	Home	3822	482369020.0500017	12096
2	Fashion	3599	463269670.4999988	11275
3	Grocery	2999	396689875.9199998	9355
4	Electronics	3120	395295560.8099997	9790
5	Beauty	2901	366172440.8300001	9089
6	null	548	67951960.89999993	1633
7	Unknown	284	37163352.42	865

7 rows | 7.914s runtime

```
--- gold_segment_sales ---
```

Table +

	segment	unique_customers	total_orders	total_revenue
1	Platinum	689	4298	547811450.3299999
2	Regular	662	4222	540057267.8199998
3	Silver	648	4110	527014138.3200002
4	Gold	657	3923	505890842.3500001
5	null	283	720	88138182.60999998

5 rows | 7.914s runtime

```
--- gold_region_sales ---
```

Table +

	region	total_orders	total_revenue	total_units
1	Online	4408	560907100.0899991	13667
2	South	3950	507050415.9600001	12365
3	North	3295	428944826.7199995	10451
4	West	3177	407807843.6800004	10074
5	East	2443	304201694.9800008	7546

5 rows | 7.914s runtime

6. Data Quality Results

The table below summarizes the outcome of Silver Stage 1 cleaning and quarantine logic across all four source entities:

Entity	Total Rows	Clean	Quarantined	Quarantine %
Orders	18,007	17,273	734	4.1%
Customers	2,500	2,500	0	0.0%
Products	800	780	20	2.5%
Stores	75	75	—	—

During Gold layer validation (Section 5.5, Figure 27), fact_orders showed a number of orders with a null customer_sk or product_sk. This was investigated rather than left unexplained:

- ~720 orders (4%) have a null customer_sk. All 720 corresponding customer_ids exist in dim_customer_scd2 - the order_date falls before the customer's earliest recorded effective_start_date (the order predates the customer's signup in the source data). A source data quality artifact, not a pipeline defect.
- ~536 orders (3%) have a null product_sk. 69 follow the same date-mismatch pattern; the remaining 16 reference product_ids that were quarantined in Silver Stage 1 due to an invalid/unparseable unit_price.

In both cases the pipeline does not silently drop these orders - they remain in fact_orders with a null surrogate key, making the issue traceable rather than hidden.

7. Delta Lake Features Demonstrated

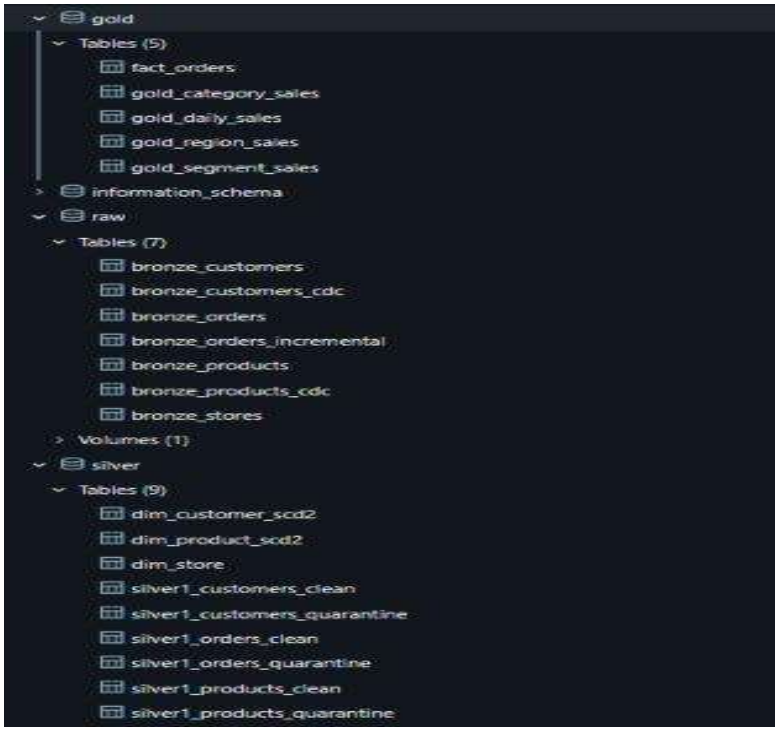
Feature	How it was demonstrated
ACID Transactions	Every write across Bronze, Silver, and Gold uses Delta's transactional writer - no partial or corrupt writes even on repeated re-runs.
Table History	DESCRIBE HISTORY retail_demo.gold.fact_orders (04_gold, Cell 5) - full transaction log with operation type, timestamp, and metrics per version.
Time Travel	fact_orders queried at VERSION AS OF 0 vs. the latest version (04_gold, Cell 6) to show table evolution.

Schema Evolution	coupon_code, introduced only in the 2026-04-26 incremental orders file, absorbed automatically via Auto Loader's addNewColumns mode (01_bronze, Cell 4) - no manual schema change, no job failure.

8. Final Deliverables

The following Delta tables were produced by running all five notebooks end-to-end on a fresh cluster, in order: 00_setup → 01_bronze → 02_silver_stage1 → 03_silver_stage2_scd2 → 04_gold.

Schema	Tables
raw	bronze_orders, bronze_customers, bronze_products, bronze_stores, bronze_orders_incremental, bronze_customers_cdc, bronze_products_cdc
silver	silver1_orders_clean / _quarantine, silver1_customers_clean / _quarantine, silver1_products_clean / _quarantine, dim_store, dim_customer_scd2, dim_product_scd2
gold	fact_orders, gold_daily_sales, gold_category_sales, gold_segment_sales, gold_region_sales



9. How to Reproduce

- Upload the datasets/ folder into a Unity Catalog volume at /Volumes/retail_demo/raw/retail_files/datasets.
- Open 00_setup and confirm BASE_PATH matches the upload location.
- Run the five notebooks in order, top to bottom, on a fresh cluster (00_setup → 01_bronze → 02_silver_stage1 → 03_silver_stage2_scd2 → 04_gold).
- Verify results under Catalog → retail_demo → raw / silver / gold, and via the sanity-check print statements included in each notebook.

10. Conclusion

This project delivers a complete, production-style retail analytics pipeline that ingests dirty real-world data, cleans and validates it without silent data loss, tracks historical dimension changes correctly using SCD Type 2, and produces a conformed fact table with dashboard-ready aggregates - all built on Databricks with Delta Lake as the storage layer. Every stage was independently verified with row counts and sample queries, and the pipeline was proven reproducible by re-running it end-to-end on a fresh cluster. The known data quality findings were investigated to root cause rather than left unexplained, reflecting the kind of data quality diligence expected in a production data engineering workflow.